RESEARCH AND IMPLEMENTATION NOTE

Four further Lean proof simplifications

DiophantineConsistency after the earlier proposals

Prepared for Vladimir Reshetnikov

26 October 2026

Reviewed repository	VladimirReshetnikov/ProveIt
Pinned revision	`9bd29079d5e1c6cc1c0bcaee6cb57d59b835cba9`
Lean version	`leanprover/lean4:v4.32.0`
Exclusion corpus	Six placed packets and five incoming ZIP archives
Deliverable	Mathematical derivations, standalone checked Lean prototypes, source and novelty evidence, reproduction scripts

Principal recommendation. Replace the exponential comparison in `main_index_gt_first` by monotonicity of the positive Pell pair recurrence. For a larger simplification, combine a natural-number perturbation estimate with exact binomial quotient recovery. These changes remove specific mathematical detours that remain after the earlier refactors.

Verification boundary. The accompanying small Lean modules are checked with the repository's pinned compiler. They establish the reusable mathematical and compilation principles. Production adapters, a full repository build, exact supplied-program preservation, net line savings, and build-time changes remain integration work. None of those unperformed steps is presented as a result.

Novelty means "not found in the directly audited six packets and five current incoming archives." It is not a claim of priority over all Lean or mathematical literature. All earlier ideas are treated as already applied.

Contents

1 Findings and scope **1**
1.1 The pinned baseline . . . 1

2 Excluding earlier ideas **1**

3 Compare Pell bases through the positive pair recurrence **2**
3.1 The current detour . . . 2
3.2 The stronger mathematical interface . . . 3
3.3 Lean boundary and payoff . . . 3

4 A natural-number perturbation kernel for both Pell upper bounds **4**
4.1 What remains duplicated . . . 4
4.2 An exact replacement for the perturbed-power estimate . . . 4
4.3 A reusable power-sandwich theorem . . . 4
4.4 The exact Pell instantiation . . . 5
4.5 Make the error tolerance a separate budget . . . 5
4.6 Integration and expected effect . . . 6

5 Recover the half-binomial value through an exact integer quotient **6**
5.1 The mathematical quantity to expose . . . 6
5.2 A checked rounding theorem with integer inputs . . . 7
5.3 How the existing source supplies the premises . . . 7
5.4 The marginal contribution after the earlier packets . . . 8

6 Define rule schedules once and derive both interpretations **8**
6.1 The duplication that finite dispatch does not remove . . . 8
6.2 A small syntax is sufficient . . . 8
6.3 What has actually been checked . . . 9
6.4 The important remaining data-preservation obligation . . . 9

7 Integration order and measurements **9**

8 Artifact guide and validation boundary **10**

9 References and exact source targets **11**

1 Findings and scope

Four directions survive a fairly aggressive exclusion test. Three concern the mathematics of the Pell argument; the fourth removes duplicated rule-schedule definitions. They are deliberately unequal in scope: a small proof replacement, a shared arithmetic kernel, direct reuse of an existing integer floor theorem, and a maintenance-oriented compiler abstraction.

	Proposal	What disappears	Assessment
1	Compare Pell bases directly	Four exponential comparisons, predecessor exponents, and small-index case work in one index theorem	Best first pilot
2	A natural perturbation and error-budget theorem	Repeated real division, reciprocal, positivity, and tolerance calculations in two upper estimates	Strongest consolidation
3	Use the existing exact binomial quotient	A real fractional-part rounding detour and repeated cross-domain adapters	Strong companion to 2
4	One rule-schedule syntax	Separate numeric and supplied-PR copies of each branch expression	Maintenance investment

The maintained directory contains 402 immediate Lean modules at this revision. This count excludes its separate `checks` directory and the top-level umbrella import. It describes the scale of the inspected tree; it does not imply that every theorem was individually audited. The concrete targets below were read in detail, and the evidence scripts record their exact bytes.

The user asks for simplifications that remain after previous proposals have been applied. Accordingly, current source lengths are used only to identify and size targets. They are not a counterfactual measurement of a repository with all previous packets integrated. A new helper's own source and adapter costs must be counted before a claim of net compression is made.

1.1 The pinned baseline

The inspected `main` head is `9bd29079d5e1`, whose commit metadata records 26 October 2026. The root `lean-toolchain` pins Lean 4.32.0. The root lockfile pins Mathlib to `81a5d257c8e4 10db227a6665ed08f64fea08e997`. The relevant public recurrence API was checked in that Mathlib revision, rather than inferred from a different release. See references [1] and [10].

All repository links in the report use the pinned revision. No changes were pushed to the repository. The archive contains proposed reusable additions and evidence, rather than a claim that the maintained development was refactored.

2 Excluding earlier ideas

The direct exclusion corpus consists of packets `2026-10-26a` through `2026-10-26f` under `research/lean-proof-simplification`, plus all five ZIPs currently in `docs/incoming`. The audit manifest records 149 placed text sources and all 400 members of the five ZIPs. Reports, source notes, novelty audits, proposed interfaces, and relevant Lean artifacts were surveyed. The archive contents were handled as inert data; none of their supplied scripts was executed.

The audit includes ideas mentioned as optional or unselected directions. This matters: an additive-equivalence treatment of aligned cells and an algebraic interpretation of cyclic word packing were already discussed in evidence notes, even though they were not prominent report

headlines. The historical `docs/incoming/README.md` was inspected for scope and routing; this report does not claim to reread every unrelated manuscript transitively linked there.

Excluded family	Why it is not proposed again
Decoder graph and fused depth/quotation proofs	Already present in several incoming packets, including the latest new-directions packet.
Signed congruence orbits and `natAbs` transport	Explicitly proposed in new-ideas and new-directions.
History folds, scanners, PR iteration, finite dispatch	Covered by the original packets, with supplied-program and cost concerns already discussed.
Circuit invariants, common tails, residual squares, depth certificates	Extensively covered, including the shared 99-node tail and generated certificates.
Witness layouts, source records, arithmetic row extraction	Existing proposals already isolate these transport and representation boundaries.
Coefficient envelopes, support bounds, finite pushforwards	Already covered as mathematical APIs and certificate schemes.
Canonical carries, explicit least/greatest cyclic carries	Already developed in additional-simplifications and new-directions.
Aligned-cell subgroup or additive equivalence	Explicitly identified in an earlier novelty audit as a refinement of old advice.
Internal constructor views, unique existence, graph fusion	Already present in the latest object-theory proposals.

The detailed `evidence/prior_art_audit.md` contains 56 substantive exclusion families, smaller local suggestions, exact archive/member pointers, and decisions on rejected candidates. `evidence/novelty_matrix.csv` records the nearest earlier proposal for each retained direction.

3 Compare Pell bases through the positive pair recurrence

3.1 The current detour

The target is `NativePellBootstrap.main_index_gt_first`, lines 309–333 of the pinned file (reference [2]). It assumes

$$c = y_A(p), \qquad k = 2y_P(n), \qquad k < c, \qquad 2A < 2P - 1,$$

as well as positive-index and lower-bound hypotheses, and proves $n < p$. Here y_A denotes the second Pell coordinate with base $A > 1$.

The proof first excludes $p = 1$, introduces $p - 1$ and $n - 1$, and chains

$$c \le (2A)^{p-1} \le (2A)^{n-1} < (2P - 1)^{n-1} \le y_P(n) \le k.$$

This works, but compares approximations to the sequences when the sequences themselves admit a simpler order comparison.

3.2 The stronger mathematical interface

Use the simultaneous positive recurrence

$$\begin{aligned} x_A(0) &= 1, & y_A(0) &= 0, \\ x_A(j+1) &= Ax_A(j) + (A^2-1)y_A(j), & y_A(j+1) &= x_A(j) + Ay_A(j). \end{aligned}$$

Lemma 3.1 (Monotonicity in the base)**.** *If $1 < A \le B$, then for every $j \in \mathbb{N}$,*

$$x_A(j) \le x_B(j), \qquad y_A(j) \le y_B(j).$$

Proof. At zero the pairs coincide. For the successor step, all coordinates and coefficients are nonnegative, $A \le B$, and $A^2 - 1 \le B^2 - 1$. Monotonicity of natural addition and multiplication transports both induction hypotheses through the displayed recurrence. □

The simultaneous form is useful. The second-order recurrence $y_A(j+2) = 2Ay_A(j+1) - y_A(j)$ contains a subtraction whose monotonicity points in the wrong direction for a direct comparison. The positive pair recurrence avoids that issue completely.

Theorem 3.2 (Index comparison from an actual value gap)**.** *Suppose $1 < A \le P$, $c = y_A(p)$, $y_P(n) \le k$, and $k < c$. Then $n < p$.*

Proof. If $p \le n$, base monotonicity and index monotonicity give

$$c = y_A(p) \le y_P(p) \le y_P(n) \le k < c,$$

which is impossible. □

The stronger helper does not need $p > 0$, $n > 0$, or $c > 1$. It uses $A \le P$ in place of the old strict exponential-base separation. The current public theorem can keep its exact signature: infer $A \le P$ from its old hypothesis, infer $y_P(n) \le k$ from $k = 2y_P(n)$, and apply the helper. This keeps downstream changes small while improving the actual proof.

3.3 Lean boundary and payoff

`lean/ArithmeticPellMonotonicity.lean` proves the pair recurrence comparison using only `Std`. It also proves an adapter for any two coordinate functions satisfying the same zero and successor equations, index monotonicity, and the gap theorem. The base-monotonicity theorem has an empty axiom list under `#print axioms`; the recurrence adapter reports `propext`.

Pinned Mathlib supplies `Pell.xn_succ`, `Pell.yn_succ`, and `Pell.strictMono_y`. Its coefficient `Pell.d` is private; an adapter can expose $A * A - 1$ by definitional equality with `change`, without referencing a private name. The exact ProveIt adapter is still to be compiled.

The immediate gain is a proof that follows the mathematical reason for the index ordering and a weaker reusable interface. The original target is only 25 physical lines, so adding a general helper may not reduce the first client's total line count. It is still the safest pilot: the transformation is local, its core is checked, and the retained wrapper makes acceptance straightforward. Deleting other base-separation lemmas would require a separate dependency check.

4 A natural-number perturbation kernel for both Pell upper bounds

4.1 What remains duplicated

`NativeHalfBinomial.upper_estimate_scaled` and `Jones1980.Ratio90.upper_estimate'` contain nearly the same growth comparison. The former allows the normalized ratio to be below $2(Y+1)$ and concludes an error below one; the latter uses $Y+1$ and concludes an error below one half. Their current theorem regions span 91 physical lines each. Those 182 lines are the target's present size, not a measured deletion budget (references [3] and [4]).

The two proofs introduce real quantities, prove positivity, clear fractions, bound a perturbed power, cancel a positive power, and only then specialize the error tolerance. Earlier advice to replace trivial existential bindings with `set` improves presentation, but leaves this mathematical duplication.

4.2 An exact replacement for the perturbed-power estimate

Theorem 4.1 (Natural perturbation)**.** *For $H, b, r \in \mathbb{N}$, if $2rb \le H$, then*

$$H(H+b)^r \le H^r(H+2rb). \tag{1}$$

Proof. Induct on r, retaining its bound as a hypothesis. At zero both sides equal H. At a successor, apply the induction hypothesis and multiply by $H+b$. The remaining comparison is

$$(H+2rb)(H+b) \le H\big(H+2(r+1)b\big).$$

After expanding and canceling common summands, this is exactly $b(2rb) \le bH$, supplied by $2rb \le H$. The successor's stronger assumption implies this bound and the induction hypothesis's assumption. All terms are natural numbers; no division or subtraction is needed. □

For $H > 0$, equation (1) is the denominator-cleared form of $(1+b/H)^r \le 1+2rb/H$. Its direct induction avoids the real reciprocal argument in the current `one_add_pow_le` proof.

4.3 A reusable power-sandwich theorem

Theorem 4.2 (Transfer the perturbation through growth bounds)**.** *Let $H > 0$. Suppose*

$$ZH \le Q(H+b), \qquad V \le Z^r, \qquad Q^r \le W, \qquad 2rb \le H.$$

Then

$$VH \le W(H+2rb). \tag{2}$$

Proof. The exact chain is

$$\begin{aligned} VHH^r &\le H(ZH)^r \\ &\le HQ^r(H+b)^r \\ &\le Q^rH^r(H+2rb) \\ &\le WH^r(H+2rb). \end{aligned}$$

Cancel the positive factor H^r. This uses (1) once. □

This theorem is useful because it exposes precisely what each application must provide. It does not package two existing proofs behind a common record. There is one induction and one comparison argument.

4.4 The exact Pell instantiation

Use the following names, with $U, Y, r \geq 1$:

$$\begin{aligned} H &= Y(U+1), & P &= 2UY^2+1, \\ D &= U^r, & T &= (U+1)^{2r}, \\ K &= y_P(r+1), & c &= y_{H+2}(2r+1), \\ Z &= (2(H+2))^2 U, & Q &= (2P-1)(U+1)^2. \end{aligned}$$

The existing natural `upper_key` theorem gives $ZH \leq Q(H+9)$ when $16 \leq H$. The existing Pell growth bounds give

$$cD \leq Z^r, \qquad Q^r \leq KT.$$

Apply (2) with $b = 9$, $V = cD$, and $W = KT$. Under $18r \leq H$, the common conclusion is

$$\boxed{cDH \leq KT(H+18r).} \tag{3}$$

Thus the shared estimate can remain entirely in $\mathbb{N}$. The existing stronger hypothesis $72r < U+1$ readily supplies the perturbation condition, but the core need not demand that stronger threshold.

For interpretation, dividing (3) by positive denominators yields

$$\frac{c}{K} - \xi \leq \frac{18r\xi}{Y(U+1)}, \qquad \xi = \frac{T}{D}.$$

This real inequality is a readable consequence of an integer certificate. It need not be the implementation interface.

4.5 Make the error tolerance a separate budget

The following natural-number implication is checked in `ArithmeticPerturbation.lean`:

$$\left.\begin{aligned} cDH &\leq KT(H+e), \\ qTe &< DH, \qquad K > 0 \end{aligned}\right\} \implies qcD < K(qT+D). \tag{4}$$

For positive q, D, K, the conclusion says that the error in c/K relative to T/D is below $1/q$. Its proof multiplies the majorant by q, multiplies the budget by K, adds the inequalities after distributing, and cancels the common factor H. The budget itself ensures the relevant product is positive.

For the old half-error estimate choose $q = 2$ and $e = 18r$. For the native estimate choose $q = 1$, then use its actual denominator $k = 2K$. The checked native wrapper concludes

$$2cD < k(T+D), \tag{5}$$

which is exactly the cross-multiplied upper inequality needed for the next section's rounding theorem.

The familiar assumption $72r < U+1$ gives

$$36r(Y+1) \leq 72rY < Y(U+1) = H.$$

Consequently $T < D(Y+1)$ supplies the budget $36rT < DH$ for the old half-error theorem, and $T < 2D(Y+1)$ supplies $18rT < DH$ for the native theorem. The latter bound on T already follows from the existing lower estimate and the source interval by multiplying and canceling positive natural numbers. No additional analytic estimate is required.

4.6 Integration and expected effect

The common kernel belongs below both consumers, beside `upper_key` or in a small module imported by `PellRatio90`. The native file already imports that module. Keep the current real-valued public theorems as compatibility wrappers, proved by one denominator-clearing conversion if clients still use their old statements.

The current `upper_estimate'` proof does not use its `hYr:U^r<=Y` parameter after the declaration. The new core can omit it; retaining it on the old wrapper preserves compatibility.

The checked artifacts establish the natural perturbation, the abstract power-sandwich transfer, and the error-budget conversion. Connecting the already available Pell growth bounds and `upper_key` to their exact parameters is still a production adapter. This is the most promising consolidation because the shared argument is visibly present in two current proofs, and the new integer interface serves both.

5 Recover the half-binomial value through an exact integer quotient

5.1 The mathematical quantity to expose

The current `half_binomial_of_ratio` obtains an integer part and a real fractional remainder from `Jones1982.rho_split_c`, proves the integer part even, compares two real intervals, then casts inequalities back to naturals to identify the integer. The target lines are 146–177 of `NativeHalfBinomial.lean`; `source_half_binomial`, at lines 193–242, builds its real arguments mainly at lines 210–241 (reference [3]).

Write $T = (X+1)^{2r}$, $D = X^r$, and $C_r = \binom{2r}{r}$. The needed natural-number certificate is already available in the imported `Jones1980.Ratio90.binomial_floor_two`. Under the same native bound on X, it returns Q, L, W with

$$T = QD + L, \qquad 4L < D, \qquad Q = C_r + WX.$$

The change is to use that existing interface directly and apply an integer rounding theorem. It requires no new development of the binomial expansion. The following derivation explains the existing certificate's mathematics; it is not presented as a new binomial identity.

For clarity, consider the exact Euclidean decomposition of the numerator. For $r \geq 1$, put

$$T = (X+1)^{2r}, \qquad D = X^r, \qquad C_r = \binom{2r}{r}.$$

Splitting the binomial sum at its middle term gives the explicit naturals

$$L = \sum_{j=0}^{r-1} \binom{2r}{j} X^j,$$
$$W = \sum_{j=0}^{r-1} \binom{2r}{r+1+j} X^j$$

and the identity

$$T = D(C_r + XW) + L. \tag{6}$$

Lemma 5.1 (The lower binomial block is a remainder)**.** *If $X > 2^{2r}$ and $r \geq 1$, then $0 < L < D$.*

Proof. The term at $j = 0$ is one. For each $j < r$, $X^j \leq X^{r-1}$, and the partial sum of binomial coefficients is at most the full sum 2^{2r}. Thus

$$L \leq 2^{2r} X^{r-1} < X X^{r-1} = X^r = D.$$

The strict inequality uses positivity of X^{r-1}, including when $r = 1$. □

The current stronger source bound $2^{2r+1} \leq X$ implies this condition. For $r \geq 1$, the identity $\binom{2r}{r} = 2\binom{2r-1}{r-1}$ makes C_r even. If X is even, then $C_r + XW = 2t$ for a natural t. Equation (6) becomes

$$\boxed{T = 2tD + L, \qquad 0 \leq L < D.} \tag{7}$$

Equivalently, $T/D = 2t$ in natural division, and its remainder is L. The existing `binomial_flo or_two` supplies a stronger remainder bound than needed here. Its $4L < D$ immediately implies $L < D$. Alternatively, `Jones1982.lemma_2_24` with its scale parameter set to one already gives the required natural decomposition. Neither existing theorem needs to be reproved. The remaining adapter applies that theorem, establishes the already-used parity fact, and connects it to the checked rounding lemma.

5.2 A checked rounding theorem with integer inputs

Theorem 5.2 (Half-quotient rounding)**.** *Let $D, k > 0$, $0 \leq L < D$, and $T = 2tD + L$. Suppose*

$$kT < 2cD < k(T + D), \tag{8}$$
$$kY < c < k(Y + 1). \tag{9}$$

Then $Y = t$, and consequently $2Y = C_r + XW$ in the binomial application.

Proof. First show $kt < c < k(t + 1)$. If $c \leq kt$, then $2cD \leq 2ktD \leq k(2tD + L) = kT$, contradicting the lower inequality. If $k(t + 1) \leq c$, then

$$2cD \geq 2k(t + 1)D > k(2tD + L + D) = k(T + D),$$

contradicting the upper inequality. The strict step uses $L < D$ and $k > 0$. Finally, two strict intervals of the form $(kj, k(j + 1))$ containing the same integer c have the same index. Apply this to $j = t$ and $j = Y$. □

`ArithmeticQuotientRounding.lean` checks this theorem for arbitrary natural inputs. It includes the interval uniqueness lemma and does not assume a Pell interpretation or any real-number theorem. The remainder is allowed to be zero, although the concrete binomial remainder is positive.

5.3 How the existing source supplies the premises

The current lower theorem gives $kT < 2cD$ directly over naturals. Proposal 2 supplies the upper half of (8) in precisely the form (5). The literal source equations give

$$c = kY + \eta, \qquad k = \eta + \zeta,$$

with positive η, ζ, and therefore (9). Once the existing binomial certificate and parity are supplied, the source theorem can apply the checked rounding result and return the same witness W.

The numerical identity being proved is unchanged. In particular, the factor two in the native first Pell coordinate remains explicit throughout. As a small exact illustration, take $r = 1$, $X = 8$. Then

$$T = 81, \quad D = 8, \quad C_r = 2, \quad W = 1, \quad L = 1, \quad t = 5.$$

With $k = 16$, $c = 85$, and $Y = 5$, both windows hold and the recovered identity is $2Y = 2 + 8$. This is an illustration of the local rounding lemma, not a claim that these values form a complete source witness.

5.4 The marginal contribution after the earlier packets

Earlier Euclidean certificates package quotient/remainder constraints in register and quadratic encodings. This proposal instead identifies the *particular integer quotient of a binomial expansion* and uses it to replace the real rounding stage. The distinction is the eliminated proof boundary, not the word "Euclidean."

Proposals 2 and 3 compose particularly well: the first ends in an integer cross-product upper bound, and the second starts with exactly that bound. They do not yet establish that the entire repository has no real-number dependencies. Their target is the local Pell-to-binomial recovery chain. The work is therefore smaller than introducing a fresh binomial-sum library. The adapter and shared rounding helper still have a cost; source reduction is an integration measurement, not a consequence of the ordinary proof alone.

6 Define rule schedules once and derive both interpretations

6.1 The duplication that finite dispatch does not remove

In `AnnotatedRuleSchedules.lean`, the numerical definitions `outputBody` and `premiseBodies` describe every rule branch. The supplied-program definitions `P.outputBodyProgram` and `P.premiseBodyProgram` repeat those expressions using realizers. The four table regions contain 63 nonblank source lines. The existing `ArithmeticFiniteSwitch.program` already handles dispatch; it cannot remove duplication inside the branch bodies (reference [7]).

The four independent bridges to the original rule calculus occupy only 13 lines together. They are `output_quote`, `premises_quote`, `ofRule_output`, and `ofRule_premises`. They should remain compact independent checks. Earlier advice already cautioned against replacing such short finite proofs with metaprogramming.

6.2 A small syntax is sufficient

The actual schedules need ten named coordinate fields: `context`, `shifted`, `a`, `b`, `c`, `instI`, `instJ`, `shiftC`, `i`, and `j`. A term language needs only field access, natural literals, atom construction, fork construction, and list cons. Binary/unary formula constructors, variable nodes, and bottom are transparent abbreviations in this language.

Store one `Schedule` per tag, consisting of an output term and a list of premise terms. For example, implication elimination has output $\mathrm{fork}(G, b)$ and premises

$$[\mathrm{fork}(G, \mathrm{binary}(3, a, b)), \mathrm{fork}(G, a)].$$

The table has all 17 valid tags and the current zero-output/empty-premises default for every invalid tag.

Numerical evaluation interprets the syntax in the existing coding operations. A backend compiles it using the actual supplied PR constructors. Its laws state that each primitive program constructor evaluates to the corresponding numerical operation. Structural induction then proves

$$\mathrm{run}(\mathrm{compile}(t), z) = \mathrm{eval}(t, \mathrm{coordinates}(z)) \tag{10}$$

for every raw input z. This theorem does not require a valid coordinate packet. Validity is used later, in the independent bridge to the original rule's output and premises.

6.3 What has actually been checked

`StructuralSchedule.lean` includes the full 17-rule table, its invalid default, independent copies of the legacy numeric tables, a generic backend, and proofs of:

- `schedule_matches_legacy`: exact numeric table agreement for every natural-number tag, including the invalid default;
- `Backend.compile_correct`: equation (10) for every term and raw input;
- `schedule_correct`: simultaneous correctness of the output and the encoded premise list.

All three have empty axiom lists. The companion Python transcription checker compares the independent legacy tables against the pinned production source after a documented renaming of field access and coding operations. Both comparisons pass. Thus the validation covers the actual table, not just a generic language unrelated to its 17 cases.

6.4 The important remaining data-preservation obligation

Instantiation with `ScalarPRCode` must use the current projection realizers and coding combinators. Before replacing maintained code, prove equality of each compiled branch's executable `.val` with the original branch tree. Equality of evaluated functions alone does not establish this.

The pinned source currently states node counts 2,537 and 2,377 for the output program before and after optimization, and 2,683 and 2,483 for the premise program. These numbers were read from source; the standalone prototype does not rerun those theorems. Replaying them is a production acceptance gate.

Generic syntax and backend definitions have a cost. This first client could grow in total source size even though the number of independent copies of each branch expression falls from two to one. The existing dependent realizer type already catches many semantic inconsistencies, so the benefit is chiefly maintenance and a common correctness argument. Adopt this after the small Pell pilot, and judge it by actual net size, change effort, and program-data preservation rather than by an assumed speedup.

7 Integration order and measurements

The following order keeps each proposed change independently reviewable.

1. Add the Pell recurrence adapter and base-monotonicity theorem. Replace the body of `main_index_gt_first` while retaining its header. Compile that file and its actual clients.
2. Add the natural perturbation and power-sandwich helpers below both upper-bound consumers. Derive the common majorant from the existing growth lemmas and `upper_key`. Recover the two existing upper-bound statements as wrappers before modifying clients.

3. Reuse `binomial_floor_two` and the existing parity fact. Combine the majorant budget, existing lower estimate, literal source interval, and checked rounding theorem in `source_half_binomial`.
4. Instantiate the schedule backend. Retain the independent bridges, prove executable-tree equality, and replay the four count theorems.

Measure production code plus new shared helpers plus adapters together. Record the number of independent inductions and cross-domain transitions as well as physical/nonblank lines. For the schedule change, record the number of independent branch descriptions and whether program trees are unchanged. Compile time and kernel time should be measured only after the exact adapters are integrated. This packet reports no benchmark or project-wide reduction.

These are implementation acceptance criteria, rather than additional new ideas. Earlier packets already discuss profiling, axiom audits, narrow simplification, and regression checks; those are assumed available.

8 Artifact guide and validation boundary

The archive is self-contained apart from an installed Lean 4.32.0 compiler and ordinary LaTeX/Python tools. It does not download a repository or install dependencies when its check scripts are run.

Artifact	Purpose
`report.tex`, `report.pdf`	Complete report and compiled rendering.
`lean/ArithmeticPellMonotonicity.lean`	Positive pair recurrence, base and index comparisons, recurrence adapter.
`lean/ArithmeticPerturbation.lean`	Natural perturbed-power inequality, abstract majorant transfer, exact tolerance selection, and native doubled-denominator wrapper.
`lean/ArithmeticQuotientRounding.lean`	Exact rounding from two integer windows.
`lean/StructuralSchedule.lean`	Actual 17 schedules, legacy-table agreement, generic compilation correctness.
`evidence/`	Full exclusion audit, candidate review, hashes, source targets, novelty matrix, and check outputs.
`scripts/`	Reproduction, source transcription, and exact arithmetic experiments.

All four standalone modules are checked with Lean 4.32.0; their compiler outputs are included. No `sorry`, `admit`, added axiom declaration, `native_decide`, or unsafe declaration is used. Some proofs report the ordinary Lean foundational axioms `propext`, `Quot.sound`, and/or `Classical.choice`; an empty axiom list is claimed only where the actual audit output says so. "No added axioms" does not mean every theorem has an empty dependency list.

The exact arithmetic experiments use integer calculations, including the binomial decomposition and representative Pell majorants. They provide additional checks on factors and signs; they do not substitute for the unbounded proofs. The source transcription check establishes correspondence with this pinned repository snapshot, not arbitrary future revisions.

The remaining formalization tasks are explicit: the actual Mathlib Pell adapter, the source instantiation of the natural majorant, the adapter from the existing binomial certificate to rounding, and the actual PR backend/data bridge. A full ProveIt build was not performed. The

report's source-size observations therefore remain target measurements and its expected benefits remain proposals.

9 References and exact source targets

[1] ProveIt toolchain, dependency lockfile, and reviewed commit.

[2] `NativePellBootstrap.lean`: `main_index_gt_first`, lines 309–333; associated classification and source-base lemmas were inspected for context.

[3] `NativeHalfBinomial.lean`: `upper_estimate_scaled`, 18–108; `doubled_upper_estimate`, 111–142; `half_binomial_of_ratio`, 146–177; `source_half_binomial`, 193–242.

[4] `Paper1980/PellRatio90.lean`: `upper_key`, 189–195; `one_add_pow_le`, 198–219; `upper_estimate'`, 222–312; and the already available `binomial_floor_two`, beginning at line 367.

[5] `NativePellPower.lean`: `doubled_ratio_lower` and power recovery. `Common/Pell.lean`: `psi_succ_le_pow` and `pow_le_psi_succ` (the source spells psi with the Greek symbol), the existing natural growth estimates.

[6] `Paper1982/Ratio.lean`: `lemma_2_24`, beginning at line 44, the existing natural certificate; and `rho_split_c`, its real-remainder interface used by the current native rounding proof.

[7] `AnnotatedRuleSchedules.lean`: numeric and supplied-program tables, the four independent bridges, and node-count declarations at lines 264–267.

[8] Placed-packet index and adoption ledger. The ledger's adopted/pending/deferred distinctions do not change this report's assumption that all earlier proposals have already been applied.

[9] Current incoming archives: `diophantine-lean-simplification-2026-10-26(1).zip`, `lean-proof-simplification-review-2026-10-26.zip`, `diophantine-lean-new-ideas-2026-10-26.zip`, `diophantine-lean-additional-simplifications-2026-10-26.zip`, and `diophantine-lean-new-directions-2026-10-26.zip`. Exact member references and hashes appear in the evidence files.

[10] Pinned Mathlib `PellMatiyasevic.lean`: positive pair definition at 104–106, `xn_succ` at 133, `yn_succ` at 137, and `strictMono_y` at 375. The file was inspected for the needed API; no exhaustive Mathlib-wide novelty claim is made.